

Auntie: Unobservable Contracts from Zerocash and Trusted Execution Environments

Adrian Cinal^[0009–0001–9789–0470]

NASK National Research Institute
`adrian.cinal@nask.pl`

Abstract. Privacy-oriented cryptocurrencies like Zerocash only support direct payments and not the execution of more complex contracts. Bitcoin and Ethereum, on the other hand, cannot guarantee privacy, and using them for contract execution leaves open questions about the fungibility of proceeds and requires contract designers to take frontrunning countermeasures. This work reconciles the two worlds and develops a practical framework for decentralized execution of complex contracts that (1) is undetectable to the network at large, (2) maintains anonymity of the potentially mutually distrustful counterparties and unlinkability of their repeated interactions, (3) guarantees fair termination, and (4) is immediately resistant to frontrunning and miner bribery attacks. This is achieved by leveraging the confidentiality and anonymity guarantees of Zerocash and the verifiability and flexibility of trusted execution environments.

Keywords: Fairness · Privacy coins · Trusted execution environments

1 Introduction

Cryptocurrencies are uniquely equipped to enforce fairness in multiparty computation (MPC) by financially penalizing offenders (who, *e.g.*, abort the protocol when the outcome would be unfavourable for them) and compensating their victims. With ownership of cryptocurrency deriving from general consensus about the state of a distributed ledger of transactions, various rules and conditions for transfer can be encoded in these transactions and subsequently enforced by the same consensus mechanism, ensuring that any transfer of money is *atomic* with respect to making good on contractual obligations. Whereas an extensive line of research [2, 3, 5, 9, 17, 19, 25, 27] has been devoted to leveraging cryptocurrencies this way, an overlooked aspect has always been the fungibility of the eventual proceeds from such “contracts.” Bentov and Kumaresan in [9], when formalizing fair MPC, assume coins to be fungible and “perfectly indistinguishable from each other,” which, say, bitcoins clearly are not due to their provenance being a matter of public record [31, 32]. In protocols studied therein, such distinguishable coins change hands in making and claiming deposits: “[t]he amount deposited equals the amount claimed” [9], but participants do not get *their* deposits back but

instead those of their counterparties. Meanwhile, coins obtained from, *e.g.*, gambling or illegal auctions could be deemed less valuable. While paying premium relative to the nominal value in order to account for future devaluations could address this, the feasibility of this approach is questionable, as quantifying the associated risk is a challenge [31,32], especially in prevailing regulatory landscape with new legislation affecting the perceived value of coins emerging yearly [35]. Transactions used for implementing contracts are also non-standard or otherwise uncommon, as they must encode (at least some of) the rules of the contract to be enforced by consensus; typically, they interact with smart contracts [5,19,21] or use features such as timelocks [2,3,9,17,27], making them clearly distinguishable from regular point-to-point payments. Furthermore, termination of (or any progress in) a decentralized contract is entirely at the discretion of miners who have the final say about when (or whether at all) to include a transaction in the ledger. This raises concerns about frontrunning [17], censorship [38], and the threat of bribery, since miners see the details of a contract *before* it has been settled. Also, historically, conservative Bitcoin miners have been observed to reject transactions with complex scripts indiscriminately [2,27]. Finally, it may be desirable for participants of a contract, say, a purchase of digital commodities, to keep the details of the purchase confidential [13] or even the very fact of it having occurred undetectable to the public. And, if the parties are distrustful of each other, they may further want to keep their identities secret from one another. All the above concerns are addressed successfully in this work.

The paper proposes a hybrid, termed AUNTIE, of *trusted execution environments* (TEEs) and the privacy-oriented cryptocurrency Zerocash [7] and develops principles of fair contract design in this setting which, despite leveraging trusted hardware, turn out to be non-trivial. AUNTIE guarantees that contracts are *unobservable* [34], meaning (1) they are *undetectable*, *i.e.*, the ledger where they have been executed and one where they have not are indistinguishable to outside parties [34], and (2) the parties remain anonymous to each other, namely, they do not learn each other’s blockchain identities (payment addresses). They are also (3) *confidential* [6], in that the parties do not, in general, learn each other’s inputs and outputs, including changes to their Zerocash balances. In particular, proceeds from such unobservable (and confidential) contracts are indistinguishable from regular payments. Finally, (4) repeated interactions between the same parties are *unlinkable* [34]. AUNTIE is maximally lightweight and only relies on (authenticated) encryption and cryptography already involved in Zerocash without, *e.g.*, additional zero-knowledge proofs or MPC (compare with, *e.g.*, [10]). With Zerocash only supporting point-to-point payments and no scripts, timelocks, multisignatures or other gadgets typically involved in decentralized contracts, the key technical challenge becomes formulating principles of contract design that have the TEEs emulate these missing pieces as well as heavyweight cryptography such as witness encryption or zero-knowledge proofs. Applying these principles with existing TEE technology poses another challenge, namely, addressing the security shortcomings of current TEE solutions, which is also elaborated in this work. While there is prior work relying on TEEs to en-

able new uses of cryptocurrency [4,8,17,20,21,26,28–30], privacy in the strongest sense considered in present paper (namely, full unobservability [34] and confidentiality [6]) has not, to the best of the author’s knowledge, been studied before.

1.1 Contributions

The paper proposes, for the first time, combining Zerocash (or any similar privacy coin technology) with trusted hardware for the purpose of contract execution and explores the advantages and pitfalls of such a hybrid. Assuming ideal TEEs, AUNTIE, the contract engine introduced in this work, enables maximally lightweight and provably unobservable [34] execution (the strongest privacy notion to date), where parties to a contract remain anonymous to each other, and the network at large cannot tell a contract was ever run, preserving the fungibility of the currency. In achieving this, a number of technical challenges are overcome posed by, among other things, the minimal assumptions on the underlying Zerocash blockchain including using Proof-of-Work puzzles as (probabilistic) proofs of contract timeout and ensuring settlement atomicity through transaction chaining. These are all of independent interest. Compared to prior work, AUNTIE offers a more comprehensive treatment of fairness that (1) accounts for pricing or valuation of commodities and (2) also addresses attacks that involve miner bribery. From a functional point of view, privacy notwithstanding, AUNTIE does not assume anything about the underlying cryptocurrency protocol except for the ability to make point-to-point payments. No support for multisignatures or even timelocks is required making the construction easily applicable to any conceivable UTXO-based privacy coin of the future. In particular, no changes to the Zerocash protocol are required allowing AUNTIE to be deployed today, provided limitations of existing TEE solutions (their not being “ideal”) are accounted for. These limitations are reviewed in this work as well, and means for addressing them and patching the resulting security vulnerabilities are proposed.

1.2 Preliminaries

Proof-of-Work Blockchains. Cryptocurrencies work by maintaining a distributed ledger that stores every transaction executed between the participants in the system. This ledger is typically organized as a sequence of bundles of transactions called *blocks* chained together using cryptographic hashes; hence the name *blockchain*. To make rewriting transaction history infeasible, a computational puzzle is associated with each block a solution to which (a *Proof of Work*) must be published alongside it for the block to be accepted to the ledger. Because “mining” blocks this way takes time (constant in expectation), any valid subchain can serve as a (probabilistic) proof of delay.¹ This is revisited later in the paper. Miners are paid fees by the transaction originators, which are, for

¹ Difficulty of the puzzle gets adjusted over time to account for changes in the mining power of the network. For simplicity, this work assumes the difficulty is constant for the duration of the execution of a contract.

simplicity, ignored for the most part in this work. Blockchains are expected to satisfy consistency, by which all participants converge on a common view of the ledger, immutability, by which a transaction once accepted remains in the ledger forever, and liveness, by which every valid transaction broadcast to the *mempool* (the staging area of transactions that miners use) is eventually included in the ledger [17]. In practice, liveness is under threat when one accounts for bribing of miners to censor certain valid transactions, an issue addressed in the present work.

Zerocash. Zerocash is a privacy-oriented cryptocurrency introduced by Ben-Sasson *et al.* in [7] and deployed as Zcash [22]. Unlike in, say, Bitcoin, Zerocash transactions reveal neither the participants nor the transacted amounts in plaintext. Instead, miners are convinced that a transaction is well-formed, *i.e.*, spends valid and previously unspent transaction outputs, and its amounts balance, through zero-knowledge proofs. Specifically, when a Zerocash note is minted, a commitment c to it (*i.e.*, to its amount and recipient address A) is added to a global accumulator. Trapdoors for this commitment are encrypted under the recipient’s public key and included in the transaction published on the blockchain. To spend this note, one must generate a (non-interactive zero-knowledge) proof of membership of the commitment c in the accumulator and of spend authority, *i.e.*, knowledge of the secret key a associated with the recipient address A committed to in c . For brevity, this work will talk about simply *signing* the input instead. (In Zcash [22], the input is indeed signed with a randomized key r and the link between r and a is what is then proved in zero knowledge.) To prevent double-spending, a *serial number* is derived deterministically from the note, revealed and proved consistent (again, in zero knowledge) in the transaction spending the note. If a serial number is ever repeated, the offending transaction is rejected by the network. Finally, every transaction contains a proof of balance, *i.e.*, that the total value of outputs minted is no larger than the total value of inputs spent.

Importantly, to efficient (probabilistic polynomial-time) adversaries, Zerocash transactions leak negligible information about who the participants are or what the transacted amount is. This enables unobservability of contracts instantiated on top of Zerocash and has important implications for their game-theoretic modelling: with parties learning no information (or, more precisely, negligible information) during contract execution, it is often possible to view the interaction as a simultaneous-move game (resistant to frontrunning). This is elaborated in present work. Finally, indistinguishability of transactions [7] makes bribing or otherwise colluding with miners difficult, as it becomes impossible to point to a single transaction that miners should censor for some reason. Instead, one likely has to convince them to stop mining altogether and account for their opportunity cost.

Trusted Execution Environments. Trusted execution environments (TEEs) are hardware components that offer a promising alternative to heavyweight cryp-

tography like zero-knowledge proofs, fully homomorphic encryption, or multi-party computation, enabling machine code to be executed with confidentiality and integrity guarantees even in the presence of a malicious operating system or hypervisor. As long as the physical CPU package of a TEE-enabled processor is not tampered with, TEEs are protected from at least software-based attacks and, to various degree, against physical attacks as well. TEEs typically also support *remote attestation* [33], a procedure through which a remote relying party can be convinced that the correct program is executing on an authentic hardware platform in a correctly established TEE, thereby extending their trust boundary to encompass a piece of hardware on a remote (counterparty’s) machine. Concretely, remote attestation involves signing a report produced by the hardware (which includes a measurement of the TEE and related metadata) with an *attestation key* endorsed by the CPU vendor. While the technology is still maturing and vulnerabilities are being actively patched, it is worthwhile to evaluate its utility and applicability to distributed protocols like cryptocurrencies [8, 16, 18, 20, 21, 23, 25, 28–30, 41].

This work assumes the standard TEE adversary model, where the attacker controls the operating system (that schedules the TEE) as well as the external interfaces of the TEE. While TEEs are supposed to guarantee data confidentiality as well as control flow and memory integrity, any interaction with the outside world, in particular, network and disk access, must be intermediated by the untrusted host. This allows for malicious data to be injected into the TEE and messages to be reordered or dropped altogether. In particular, the TEEs cannot rely on their hosts to give them accurate view of the blockchain, as it may be in their best interest to isolate the TEE, by itself having no meaningful sense of time, and schedule it off the CPU for sufficiently long to present it with a forged subchain. (Notably, the host will not be able to forge specific *transactions*, as these are digitally signed, but may try convincing the TEE that a transaction was *not* published.) Similarly to prior work [17, 29, 36, 42], this work views vulnerabilities of the underlying TEE technology a subject of separate research orthogonal to the study of protocols built on top of TEEs and assumes, for the most part, that they are ideal, namely, perfect black-box devices [33] that leak no data across their interface via side channels and guarantee perfect control flow integrity resistant to fault injection (*cf.* [23]). Memory integrity and confidentiality are also assumed. This “UC-style” [33] approach is justified by having the design explored in this work be TEE vendor-agnostic. In particular, it can be implemented using potentially next-generation TEEs. The impact of known vulnerabilities, however, should they be exploited, is assessed in Section 4 which details which fairness and privacy guarantees are upheld if physical protections fail and the “black box” assumption is violated.

2 Related Work

Most relevant for this work is that of Das *et al.* [17], who propose FASTKITTEN: a TEE-based contract execution framework where a single *operator* hosts a TEE

and is held accountable by a timelocked *penalty transaction*. The requirement for timelocks support prevents FASTKITTEN from being applied *as is* to a privacy-oriented cryptocurrency like Zerocash. Present work, however, can be viewed as extending FASTKITTEN to the setting of Zerocash by, among other things, replacing penalty transactions with different mechanisms enforcing honest behaviour. Bowe *et al.* in [10] propose ZEXE: a contract execution framework extending Zerocash that hides both inputs to a contract and the functionality implemented by it using zero-knowledge arguments. Present work can be viewed as proposing and evaluating a significantly more lightweight alternative to ZEXE that can additionally be immediately deployed as an overlay on top of the existing Zerocash ledger [22]. Given that established systems like Zerocash have associated with them strong inertia and network effects, phasing them out in favour of a contract-capable privacy coin does not seem feasible (*cf.* [1] and also [17]), making immediate applicability an important advantage of AUNTIE over ZEXE. Castejon-Molina *et al.* in [13] construct contingent payments that are unlinkable by virtue of relying on coin mixers. Present work offers much stronger unlinkability due to reliance on Zerocash. It also replaces external intermediaries with TEEs hosted directly by the parties to a contract. Thyagarajan *et al.* in [37] stress the importance of preserving fungibility formalized as indistinguishability of contract execution from regular point-to-point payments. Wüst *et al.* in [42] are the first, to the best of the author’s knowledge, to propose using TEEs in conjunction with Zerocash but do it only to offload blockchain monitoring and enable lightweight clients. Privacy of contracts settled on blockchains is further considered in [5, 12, 13, 24, 26]. Full unobservability is outside the scope of all of them. Finally, the synergy between TEEs and blockchains is also explored in a number of other works, including [8, 16, 18, 20, 21, 23, 25], and systematized in [41].

3 Unobservable Contracts

For the purposes of this work, a *contract* is a protocol realizing a (randomized) functionality $\mathcal{F}$ that takes, from each *player* $i = 1, \dots, n$, an input $x_i \in \{0, 1\}^*$ (potentially a “commodity” of some economic value [19]) as well as a *deposit* of $d_i \in \mathbb{N}$ coins and outputs to each player a string $y_i \in \{0, 1\}^*$ (possibly, again, of some economic value) and pays them $p_i \in \mathbb{N}$ coins. Assume each player hosts a TEE. Furthermore, as in FASTKITTEN [17], assume there exists a distinguished party, called the *operator*, who also hosts a TEE and that will *clear* the contract. Specifically, the operator’s TEE will evaluate $\mathcal{F}$ and create a settlement transaction t that will make the payments p_i accordingly. This operator can be one of the players, just as in FASTKITTEN, or can be an external party paid for their service by the players in fees [17]. For simplicity, contracts are assumed to be stateless, which includes applications of practical interest such as auctions, lotteries and fair exchange (*cf.* [17, 19]).² Extending the principles developed in

² Note that, while payment channels do not satisfy the above definition of a contract, the principles developed in this paper can naturally be leveraged to bestow unobservability on the construction of [29] as well.

this work to contracts that maintain state across invocations is left as direction for future research (*cf.* [23, 25]). Suppose the TEEs of all parties (*i.e.*, the players and the operator) have attested in front of all other parties to the code (implementing the functionality $\mathcal{F}$, refund logic, *etc.*) they are executing, and that every player i 's TEE has also established a secure (*i.e.*, encrypted and authenticated) channel to the operator's TEE with the associated shared key k_i . Notably, the keys k_i are known only to the TEEs and *not* their hosts.

We will have each player i 's TEE generate a *deposit address* A_i^{dep} with the associated spending key a_i^{dep} . The operator's TEE will also establish a deposit address A_0^{dep} with secret spending key a_0^{dep} . The parties will make their deposits (d_i s for the players³ and some *collateral* for the operator) to the respective deposit addresses A_i^{dep} established by the TEEs they host. If the contract gets aborted prematurely, the keys a_i^{dep} to these addresses will be released to the parties effectively refunding them. Each party (player or operator) will confirm (obviously) that the remaining parties have published their deposits. The operator's TEE will then evaluate the functionality $\mathcal{F}$, escrow the results y_i with the players' TEEs, and issue a settlement transaction t to its host that effectuates the payments p_i and makes the operator whole. The operator will be incentivized to publish t so they can redeem the collateral they have staked. Other players will then be able to use t they have read from the blockchain to convince their TEEs to release the outputs y_i to them. Details are elaborated in Section 3.3.

No external trusted or semi-trusted parties are assumed such as reliable time servers [15] or arbiters for dispute resolution [19], although, for the purposes of remote attestation of TEEs, access to some public-key infrastructure of the CPU vendor may be needed (*cf.* [33]).

3.1 Security Requirements

This work adopts the notion of fairness from [17] and extends it with ideas from [19] to account for the utility players get from learning the outputs of the contract (even at some financial cost) and the disutility they suffer from other players' learning the outputs unfairly. Intuitively, it should be impossible for one party to default on their obligations but nonetheless benefit from the contract or indeed for any party to unfairly cause losses for other parties. A contract is therefore *fair* if, except with negligible probability and when there is at least one honest party, it either (1) terminates correctly, (2) gets aborted before the settlement transaction t has been issued with all honest parties staying financially neutral and no outputs y_i being revealed to any player, or (3) gets aborted after t has been issued with all honest players staying financially neutral, and *all* dishonest parties (the operator, in particular, and any players colluding with them) losing money. Compare this with FASTKITTEN [17] which only requires that *at least one* dishonest party be punished. Also, note how the fairness definition above subsumes the notion of *operator balance security* from [17], which requires that an operator acting honestly cannot lose their collateral.

³ Plus extra fees to compensate the operator if needed [17].

Say a contract is *undetectable* [34] if no efficient distinguisher, themselves not a party to the contract, can tell apart two ledgers: one on which the contract was executed and one on which it was not. A contract is *unobservable* [34] if it is undetectable, and each player i only learns negligible information about the ledger identities (addresses) of other players. It is furthermore *unlinkable* [34] if two repeated interactions between the same parties cannot be linked together except with negligible advantage and *confidential* [6] if parties learn negligible information about each others' deposits d_i , payments p_i , inputs x_i , and outputs y_i in the course of its execution (except what is revealed by the evaluation of $\mathcal{F}$ itself). Note how, say, in a blind auction setting, true bid secrecy cannot really be achieved without using a cryptocurrency like Zerocash that hides transaction amounts: in previous designs [17], even if the exact bid was not revealed (*e.g.*, it was hidden by a cryptographic commitment or known only to a TEE), an upper bound on it was nevertheless known, as this was what a bidder had deposited in the (smart) contract. This crude frontrunning countermeasure also meant that parties had to overdeposit and lock up capital only to hide their bidding strategy. In this paper, no funds need to be locked by the parties except the actual bids $b_i = d_i$.

The AUNTIE framework developed in the remainder of this work is going to lead to the following result:

Theorem 1. *Let $\mathcal{F}$ be a functionality as above. Assuming trusted execution environments realizing the TEE ideal functionality [17, 33], IND-CCA-secure encryption ($\text{Gen}, \text{Enc}, \text{Dec}$), and a blockchain realizing the Zerocash ideal functionality, there exists a fair, unobservable, unlinkable, and confidential contract for $\mathcal{F}$.*

Explicit definition of the Zerocash ideal functionality is omitted, as it is virtually the same as the BC ideal functionality in [17], except it additionally accounts for the privacy properties of Zerocash, namely, *ledger indistinguishability* [7]. Also, note that the TEE ideal functionality as formalized by Pass *et al.* [33] assumes anonymous (and unlinkable) remote attestation, an issue elaborated below.

3.2 Technical Challenges

Before introducing AUNTIE, let us survey some of the technical challenges needed to be overcome when deploying unobservable contracts intermediated by TEEs, specifically, challenges to fairness and privacy arising from relying solely on point-to-point payments and TEEs that can be disconnected (denied service) by their hosts. These stem not from the imperfections or limitations of existing TEE solutions but are inherent to the distributed nature of contract execution and are all addressed successfully by AUNTIE. Problems related to TEE compromise through, *e.g.*, side channels are discussed in Section 4.

Anonymous Attestation. While, in prior work [4, 8, 15–18, 20, 21, 25, 28–30], the remote attestation process was taken for granted, it becomes of concern when

striving for full anonymity and, in particular, unlinkability. Indeed, remote attestation involves signing a hardware-generated report with a key that is derived from a hardware root of trust and certified by the CPU vendor, *e.g.*, Intel. So as to prevent linking interactions with the same CPU (and hence, likely, the same counterparty), group signatures can be used for attestation as originally done by Intel [11] (before the cloud, Intel SGX was intended for client devices to, *e.g.*, enforce DRM policies; hence, anonymity of end users was a priority), or the CPU can be reprovisioned with a new attestation key for each contract.⁴ Note that anonymous attestation is the default in formal (UC) modelling of TEEs [33].

Commodity Escrow and Proof of Settlement. Player i 's output y_i must be escrowed with their TEE before the settlement transaction t is issued, so that the operator cannot shut down their TEE and abort the protocol immediately after t has been published, leaving i without y_i . Player i can use t to convince their TEE the contract has been settled, since t contains, in particular, a signature produced with the key a_i^{dep} believed to be encumbered within the TEE.

Verifiable Delay and Refunds. If counterparties have defaulted on their obligations and aborted the contract, honest parties should have a way of exiting the contract gracefully and redeeming their deposits. A standard way to achieve this is to design contracts with *timeout* [2,3,17,19,37]. If some amount of time has passed without settlement, the parties should be issued refund transactions.⁵ With TEEs not having any meaningful notion of time [15,16], their hosts must prove timeout using, *e.g.*, verifiable delay functions or, a more practical approach, Proof-of-Work puzzles [16,25]. If timeout is expressed in a number of blocks τ , then player i can present a subchain of length τ appended to a *checkpoint block* [17] agreed upon at setup to convince their TEE that the contract was aborted. The Proof-of-Work puzzles function as verifiable (probabilistic) delay with the extra property that an honest player does not need to invest any computational resources to produce the proof: it is instead a by-product of the miners maintaining the blockchain and proves that *at least* the time needed by honest miners to produce τ blocks has elapsed.⁶ This approach, importantly, does *not* protect against player i 's selfish mining: given enough time, i can and *will* produce a subchain of length τ , all while their TEE is isolated from the others. This is fine if the settlement transaction t gets published asynchronously while i is selfishly mining the fraudulent subchain: the key a_i^{dep} eventually released by i 's TEE (or any refund transaction issued by it) will be worthless as it

⁴ See, *e.g.*, https://cc-enabling.trustedservices.intel.com/intel-tdx-enabling-guide/02/infrastructure_setup (accessed: 30 March 2026).

⁵ Without loss of generality, we can assume no TEE will release both the output y_i and a refund transaction.

⁶ This assumes that no coalition of players can ever outpace honest miners. However, one must assume this anyway to discount the threat of blockchain reorganization and for the notion of fairness (or, indeed, even paying anyone) to be meaningful.

will only allow attempting double-spending relative to t . What remains a concern is that i will identify t in the mempool, learn their payout, thus inferring something about the outcome of $\mathcal{F}$ they may not be happy with, and bribe the miners to censor t for long enough to claim timeout and unfairly back out of the contract. This issue is addressed next.

Settlement Transaction Obliviousness. As hinted above, targeting the settlement transaction t enables unfairly⁷ backing out of the contract if unhappy with the result. This is addressed in AUNTIE by intermediating the payments p_i with additional *payout addresses* A_i^{pay} . Instead of t crediting the players directly, it credits the addresses A_i^{pay} under the control of the TEEs and *not* their owners (*cf.* [4]). Zerocash’s confidentiality and receiver anonymity guarantees [6, 7] ensure that no player i will be able to tell, after only seeing t in the mempool, if (and how much) it credits their corresponding address A_i^{pay} .⁸ They can only learn this through their TEE and, when they do, they will have forfeited their right to claim a refund (see Figure 1). This way, any decision to abort the contract must be uninformed. Additionally, to get the output y_i and the spending key a_i^{pay} corresponding to the address A_i^{pay} , require that player i present their TEE with t buried under some number $\tau' > \tau$ of valid blocks.⁹ Without this, a dishonest operator could, upon learning t , disclose it to, say, player i colluding with them, only for player i to claim y_i early and, based on it, decide if they want the contract to be finalized or bribe the operator to withhold t indefinitely. Now, if obtaining y_i takes longer (τ' blocks) than honest parties’ claiming timeout and getting refunded (τ blocks), which, notably, *invalidates* the settlement transaction t (it would be double-spending relative to the refunds), the operator’s decision to withhold t must be uninformed: any party only learns the details of the settlement transaction if the contract terminates fairly or after it has timed out. Player i colluding with the dictator can still learn y_i unfairly but is then guaranteed to be financially punished if they prevent t from being published, since their TEE will not refund them anymore (see Figure 1).

Settlement Atomicity and Chaining. It is important a single transaction t settle the contract for *everyone* atomically, denying the operator the possibility of redeeming the collateral of Δ without crediting all beneficiaries of the contract. For a small number of players n , one can bundle all the payouts in a single transaction, leveraging the fact that Zerocash transactions are not malleable [22] and signed as a whole with all their outputs fixed. As n grows larger, however, limits on transaction size become a problem. Also, transactions with a

⁷ Technically, they would suffer financial losses due to the bribes issued, thereby satisfying the standard notion of fairness [17]. Nevertheless, since the cost of the bribes is external to the contract and hence difficult to account for, AUNTIE addresses this issue explicitly.

⁸ Zero-valued “dummy” outputs can be used to hide the *number* of real outputs [22].

⁹ We assume here that no coalition of parties can selfishly mine τ blocks before the network mines τ' .

large number of outputs “stand out” affording a distinguisher between contract execution and regular blockchain activity as well as enabling targeting t (still in the mempool at that point) in a bribery attack as consider earlier. This paper proposes that this be resolved using the following *chaining* technique. If the operator’s collateral is redeemed in a transaction t , their TEE can force them to publish another transaction t' by adding an output to t' that is then immediately spent in t . The two transactions are then “chained,” namely, t will not be appended to the blockchain, unless t' has been appended first. This is reminiscent of a well-known *child-pays-for-parent* technique from Bitcoin, whereby users incentivize miners to mine a low-fee transaction by chaining another high-fee transaction to it. Exactly studying fairness guarantees in this setting under operator-player collusions is left for future work.

3.3 The Auntie Framework

Suppose the setup procedure has been performed involving remote attestation of all TEEs, establishment of encrypted and authenticated channels between TEEs secured with keys k_i , as well as agreeing on a checkpoint block [17] relative to which timeout will be proved, two periods (expressed in a number of blocks) τ and τ' , and the operator’s collateral amount Δ . The pseudocode for the players’ and the operator’s TEEs is presented in Figures 1 and 2. Maintenance of a state machine that enforces the TEE interface is not abused and calls are made in proper order is omitted for clarity of exposition. All calls proceed in the order explained below except for the calls to the Refund methods which may be called at any time during contract execution.

A number of subroutines are used. **CreateWallet** creates a Zerocash *wallet* (a, A) , *i.e.*, a pair comprising an address A and the associated spending key a . **DepositedAmount**, on input a transaction t and spending key a , sums the amounts that t credits to the corresponding address A . **CreateTx**, on input a set of transactions (UTXOs) and output descriptions of the form $\{(p_j, A_j)\}_j$, creates a (yet unauthorized) transaction that spends its inputs and, for each j , credits address A_j with p_j coins. **AuthTx** attaches signatures to a yet unauthorized transaction. **BlocksSinceCheckpoint**, on input a chain of blocks B , verifies the blocks are all chained correctly, have valid Proof-of-Work puzzle solutions, and are built on top of the pre-agreed (*e.g.*, hardcoded) checkpoint block. It then returns the length of B . **AuthdAndBuried**, on input an unauthorized settlement transaction $\hat{t}$, spending key a , and a chain of blocks B , verifies the blocks are all chained correctly, have valid Proof-of-Work puzzle solutions, are built on top of the checkpoint block, and, in particular, contain a transaction t that is the “authorized version” of $\hat{t}$ and includes a signature under a . It then returns the number of blocks in B that have been appended to the block containing t . Both **BlocksSinceCheckpoint** and **AuthdAndBuried** return, say, -1 when their inputs are invalid. Failed assertions are assumed to terminate the call with a return value of $\perp$.¹⁰

¹⁰ The pseudocode neglects operator’s fees which are straightforward to account for. Also, when the operator is one of the players, say, player i , the number of transactions

Player(<i>i</i>).Initialize	Player(<i>i</i>).ConfirmDeposits(<i>b</i>)
1 : $(a_i^{\text{dep}}, A_i^{\text{dep}}) \leftarrow \text{CreateWallet}()$	1 : $\sigma_i \leftarrow \text{Sign}_{a_i^{\text{dep}}}(\hat{t})$
2 : $(a_i^{\text{pay}}, A_i^{\text{pay}}) \leftarrow \text{CreateWallet}()$	2 : return $\text{Enc}_{k_i}(b, \sigma_i)$
3 : return A_i^{dep}	
Player(<i>i</i>).DepositAndInput(<i>t_i</i>, <i>x_i</i>)	Player(<i>i</i>).Settle(<i>B</i>)
1 : $d_i \leftarrow \text{DepositedAmount}(t_i, a_i^{\text{dep}})$	1 : assert ($\text{AuthdAndBuried}(\hat{t}, a_i^{\text{dep}}, B) \geq \tau'$)
2 : return $\text{Enc}_{k_i}(t_i, x_i, d_i, A_i^{\text{pay}})$	2 : $a_i^{\text{dep}} \leftarrow \perp$
	3 : return (a_i^{pay}, y_i)
Player(<i>i</i>).GetDeposits(<i>c</i>)	Player(<i>i</i>).Refund(<i>B</i>)
1 : $(\hat{t}, y_i, t_0, t_1, \dots, t_n) \leftarrow \text{Dec}_{k_i}(c)$	1 : assert ($\text{BlocksSinceCheckpoint}(B) \geq \tau$)
2 : return $(t_0, t_1, \dots, t_n)$	2 : $(a_i^{\text{pay}}, y_i) \leftarrow \perp$
	3 : return a_i^{dep}

Fig. 1: AUNTIE pseudocode for the player's TEE

Contract execution begins with each player i 's call to **Player(*i*).Initialize**, which establishes the deposit and payment wallets and gives the player a deposit address A_i^{dep} . Each player i then creates and broadcasts a transaction t_i that credits address A_i^{dep} with d_i coins. Both t_i and x_i are input to player i 's TEE via the **Player(*i*).DepositAndInput** interface. The TEE then sends (via the respective untrusted hosts, *i.e.*, player i and the operator) a tuple $(t_i, x_i, d_i, A_i^{\text{pay}})$ to the operator's TEE which has been initialized concurrently with a call to **Operator.Initialize** that gave the operator a deposit address A_0^{dep} . The operator then creates and broadcasts a collateral transaction t_0 that credits A_0^{dep} the collateral amount Δ . They then call **Operator.ClearContract** with their own (externally created) address A_0^{pay} , the collateral transaction t_0 , and the messages (encrypted) collected from the player's TEEs. The TEE decrypts each message to learn each deposit transaction t_i , its amount d_i , the input x_i , and the payout address A_i^{pay} . (Any decryption errors abort the contract.) The amounts d_i and the inputs x_i are then used to evaluate the functionality $\mathcal{F}$. The yet unauthorized (*i.e.*, without the necessary signatures) settlement transaction $\hat{t}$ is created (formatted) at this point and (a hash thereof) sent to each player i 's TEE to be signed with a_i^{dep} in a *CoinJoin*-like fashion, where each player authorizes their corresponding input to be spent. The deposit transactions t_i including the operator's collateral transaction t_0 are also sent together with the outputs y_i to the players' TEEs at this point. The operator is given all the deposit transactions t_i by their TEE

published can be reduced by bundling the deposit of operator's collateral with the transaction t_i .

Operator.Initialize	Operator.ClearContract ($A_0^{\text{pay}}, t_0, c_1, \dots, c_n$)
1 : $(a_0^{\text{dep}}, A_0^{\text{dep}}) \leftarrow \$ \text{CreateWallet}()$	1 : assert ($\text{DepositedAmount}(t_0, a_0^{\text{dep}}) = \Delta$)
2 : return A_0^{dep}	2 : foreach $i \in \{1, \dots, n\}$ do
	3 : $(t_i, x_i, d_i, A_i^{\text{pay}}) \leftarrow \text{Dec}_{k_i}(c_i)$
	4 : endforeach
Operator.Finalize ($c_1, \dots, c_n$)	5 : $((y_1, p_1), \dots, (y_n, p_n))$ $\leftarrow \$ \mathcal{F}((x_1, d_1), \dots, (x_n, d_n))$
1 : foreach $i \in \{1, \dots, n\}$ do	6 : $\hat{t} \leftarrow \$ \text{CreateTx}((t_0, t_1, \dots, t_n),$ $(\Delta, A_0^{\text{pay}}), (p_1, A_1^{\text{pay}}), \dots, (p_n, A_n^{\text{pay}}))$
2 : $(b, \sigma_i) \leftarrow \text{Dec}_{k_i}(c_i)$	7 : foreach $i \in \{1, \dots, n\}$ do
3 : assert ($b = 1$)	8 : $c'_i \leftarrow \$ \text{Enc}_{k_i}(\hat{t}, y_i, t_0, t_1, \dots, t_n)$
4 : endforeach	9 : endforeach
5 : $\sigma_0 \leftarrow \$ \text{Sign}_{a_0^{\text{dep}}}(\hat{t})$	10 : return $(t_1, \dots, t_n, c'_1, \dots, c'_n)$
6 : $t \leftarrow \$ \text{AuthTx}(\hat{t}, \sigma_0, \sigma_1, \dots, \sigma_n)$	
7 : $a_0^{\text{dep}} \leftarrow \perp$	Operator.Refund (B)
8 : return t	1 : assert ($\text{BlocksSinceCheckpoint}(B) \geq \tau$)
	2 : $\hat{t} \leftarrow \perp$
	3 : return a_0^{dep}

Fig. 2: AUNTIE pseudocode for the operator's TEE

to verify they have been broadcast to the network and mined. Recall that Zerocash transactions reveal no (non-negligible amount of) information about the amounts deposited or who originated them, so they can be safely handed to the operator to look for them on the blockchain. Similarly, player i is given t_0 and the deposit transactions t_j of the remaining players by their TEE (via a call to $\text{Player}(i).\text{GetDeposits}$) to confirm they all have been added to the blockchain. If they have, player i calls $\text{Player}(i).\text{ConfirmDeposits}$ with $b = 1$ which results in their TEE's signing $\hat{t}$ and sending the signature to the operator's TEE. Up until this point, any foul play results in the protocol's halting with all parties eventually being able to redeem their deposits by proving contract timeout to their TEEs (see $\text{Player}(i).\text{Refund}$ and Operator.Refund in Figures 1 and 2, respectively). If happy with the deposits $t_1, \dots, t_n$, the operator calls Operator.Finalize with the (encrypted) signatures on $\hat{t}$ received from the players' TEEs. Past this point, the operator's TEE will no longer accept claims of timeout, *i.e.*, the operator cannot back out of the contract now. If they default and refuse to broadcast t , the collateral Δ is lost (potentially together with any fees paid by the players to compensate the operator for their service, see [17]), thus punishing the cheating operator. Otherwise, it is paid back to them in t .

Players wait for t to be published and periodically input the suffix of the blockchain, starting from the checkpoint block, to their TEEs (via a call to the $\text{Player}(i).\text{Settle}$ interface). At some point, t will have been mined and buried under τ' blocks, at which point the TEEs will be convinced the contract has been settled and release the outputs y_i to their hosts together with the keys a_i^{pay} . If $\tau < \tau'$ blocks were instead appended to the checkpoint block that do not contain the settlement transaction t , each player's TEE will issue a refund that pays the deposit d_i back to player i . This can be done by releasing the spending key a_i^{dep} for the i th deposit wallet.

Lemma 1. *Assuming trusted execution environments realizing the TEE ideal functionality [17, 33], IND-CCA-secure encryption ($\text{Gen}, \text{Enc}, \text{Dec}$), and a blockchain realizing the Zerocash ideal functionality, AUNTIE is fair.*

(Note that we should view Sign as an interface of the Zerocash ideal functionality, which in particular implies that signatures are unforgeable.)

Proof (sketch). Any aborts before Operator.Finalize is successfully called result in every honest party (player and operator alike) claiming and successfully proving timeout after τ blocks, and the outputs y_i , if even computed in the first place, remain escrowed with the TEEs and are never revealed to the players. Once Operator.Finalize has returned, the operator must act honestly and publish t lest they suffer a loss of Δ coins. Furthermore, if the operator does not publish t but sends it out of band to, say, player i , then player i can, having selfishly mined τ' blocks, convince their TEE to release y_i , but, by that time, a_i^{pay} will be worthless, as honest players will have long since claimed timeout (recall that $\tau < \tau'$), and a_i^{dep} will have been destroyed in the call to $\text{Player}(i).\text{Settle}$ (see Figure 1), thereby punishing player i . The same holds for *all* players colluding with the (malicious) operator.

Lemma 2. *Assuming trusted execution environments realizing the TEE ideal functionality [17, 33] and a blockchain realizing the Zerocash ideal functionality, AUNTIE is unobservable and unlinkable. Furthermore, if Enc is IND-CPA-secure, then AUNTIE is confidential.*

Proof (sketch). Intuitively, the ledger indistinguishability property of Zerocash [7] guarantees that contracts built on top of it are undetectable. Anonymity and confidentiality are achieved in AUNTIE by virtue of intermediating all deposits and payments by TEEs hosted by the players themselves and also escrowing inputs with them. When coupled with network-layer obfuscation (*e.g.*, onion routing) and unlinkability of TEE remote attestation, the above hybrid makes the parties to a contract anonymous to each other and their repeated interactions unlinkable.

Together, Lemmata 1 and 2 imply Theorem 1. Observe also that Lemma 1 does not rely on any privacy properties of the underlying ledger (hence the weaker BC ideal functionality of [17] could be used instead of the Zerocash

ideal functionality). Zerocash ledger indistinguishability [7] is only needed for Lemma 2. Finally, observe how the unobservability of AUNTIE additionally mitigates threats to liveness stemming from miner bribery and censorship.

4 TEEs of Today and Their Limitations

Trusted execution environment technology, however promising, is currently still very flawed (see, *e.g.*, [40]). This section surveys classes of vulnerabilities of TEEs and the scope of their impact on the properties of AUNTIE.

Side-channel leakage can largely be mitigated by relying on constant-time software with data-independent memory access patterns [23] with such hardening being standard practice by now, at least for cryptographic algorithms such as signing or decryption. Notably, much of the secret data of the TEEs is only ever used once or twice, minimizing its exposure. *E.g.*, the output y_i , once computed, is only touched during encryption operator-side and then decryption player-side. Likewise, the spending key a_i^{dep} , once generated, is only used once to sign the settlement transaction, and the key a_i^{pay} is only used to derive A_i^{pay} . Relying on separate deposit addresses A_i^{dep} for each player and cooperatively signing the settlement transaction t (with each TEE signing its corresponding input) also ensures that player i exploiting any side channels can at best leak a_i^{dep} and “steal” their own coins back. The settlement transaction will then be invalid and remaining players will be able to claim timeout. To protect the operator from losing their collateral of Δ as well, they may be given the serial numbers of the deposit outputs ahead of time to be able to tell if TEE guarantees were breached and the deposits redeemed ahead of time. Side channels may thus violate confidentiality of the contract (when exploited by the operator) but have little to no bearing on its fairness as considered by Das *et al.* [17] or operator balance security. Clearly, unobservability of the contract and unlinkability of different sessions hold regardless of the data leaked during contract execution. Similarly, parties remain anonymous by virtue of the per-contract intermediate addresses A_i^{dep} and A_i^{pay} used. The operator’s biasing the randomness used to evaluate $\mathcal{F}$ or produce $\hat{t}$ can be prevented by having the TEE rely on a pseudorandom number generator seeded with contributions from every player. Fault injection attacks (discounted altogether by, *e.g.*, [36] as “confined to a lab”) may allow “skipping” branch statements and bypassing controls. Repeating computations and aborting on any inconsistency may be used to mitigate these. Finally, full compromise of the TEE through extracting the attestation keys [14, 39] enables parties to forge attestation reports and fool others into escrowing both coins and inputs x_i with an untrusted program merely simulating a TEE. If player i ’s TEE is breached, then they may learn their output y_i unfairly but nevertheless cannot steal other players money, as the settlement transaction t must be signed by *all* TEEs. If the operator’s TEE is breached instead, no guarantees about either privacy or fairness hold, except that the players’ deposits remain safely escrowed with *their* TEEs (compare with [17] where compromise of the operator’s TEE voids *all* guarantees).

5 Implementation

AUNTIE has been implemented using Intel SGX. The source code is available at <https://github.com/aCinal/auntie>.

6 Conclusions and Future Work

The hybrid of trusted execution environments and Zerocash has not been explored before in the context of contract execution. As the AUNTIE framework introduced in this work demonstrates, it offers an elegant solution that simultaneously addresses fairness, privacy, fungibility, as well as frontrunning and bribery prevention. It enables mutually distrustful parties’ engaging in contracts without learning anything (non-negligible) about each other’s identities while leaving no traces of the contract on the Zerocash blockchain and, in particular, ensuring the proceeds from the contract are indistinguishable from funds obtained through direct payments. The hybrid also lends itself to a game-theoretic treatment with privacy guarantees of Zerocash ensuring players gain no insight during contract execution, thus reducing many sequential-move games to simultaneous-move ones, where the players may as well decide on all choices *a priori*.¹¹

When compared to ZEXE [10] (whose goals are distinct but nonetheless overlap with those of present work), AUNTIE offers vastly superior performance and is, most importantly, immediately applicable to Zerocash as deployed today [22]. This is particularly relevant, since, as observed before [1, 17], replacing established designs may not be easy due to strong network effects of cryptocurrencies. When compared to FASTKITTEN [17], AUNTIE provides superior privacy including full undetectability of the contract execution, anonymity of participants, unlinkability of their interactions, and confidentiality of deposits (on top of confidentiality of inputs x_i and outputs y_i already supported by FASTKITTEN).

An interesting direction for future work is formalizing the conditions under which optimizations of the general framework are possible, *e.g.*, by relaxing the assumption that *every* player host a TEE, or that the operator make a deposit. For example, fair exchange (contingent payment) [19] can be achieved with only the buyer hosting a TEE that the seller escrows the commodity with and from which the seller also receives the settlement transaction t . It must, however, be the seller that receives t (notably, from the buyer’s TEE), as only this allows the buyer’s proving the contract has been settled to their TEE. There is clearly no single “operator” in this example, but readers are encouraged to convince themselves that the construction is nonetheless fair.

¹¹ This is assuming perfect envelopes for encryption and perfect zero-knowledge where applicable. Accounting for negligible leakage due to working under computational assumptions is outside the scope of this work.

References

1. Anderson, R., Shumailov, I., Ahmed, M.: Making Bitcoin legal. In: Matyáš, V., Švenda, P., Stajano, F., Christianson, B., Anderson, J. (eds.) *Security Protocols XXVI*. pp. 243–253. Springer International Publishing, Cham (2018)
2. Andrychowicz, M., Dziembowski, S., Malinowski, D., Mazurek, : Secure multiparty computations on Bitcoin. *Cryptology ePrint Archive* **2013**, 784 (2013), <https://eprint.iacr.org/2013/784>
3. Andrychowicz, M., Dziembowski, S., Malinowski, D., Mazurek, : Fair two-party computations via Bitcoin deposits. In: *Proceedings of the 2014 Financial Cryptography and Data Security*. pp. 105–121. Springer (2014), https://www.ifca.ai/fc14/bitcoin/papers/bitcoin14_submission_10.pdf
4. Austgen, J., Fabrega, A., Kelkar, M., Vilardell, D., Allen, S., Babel, K., Yu, J., Juels, A.: Liquefaction: Privately liquefying blockchain assets. In: *2025 IEEE Symposium on Security and Privacy (SP)*. pp. 1493–1511. IEEE Computer Society, Los Alamitos, CA, USA (May 2025). <https://doi.org/10.1109/SP61157.2025.00156>, <https://doi.ieeecomputersociety.org/10.1109/SP61157.2025.00156>
5. Avizheh, S., Haffey, P., Safavi-Naini, R.: Privacy-preserving FairSwap: Fairness and privacy interplay. *Proceedings on Privacy Enhancing Technologies* **2022**(1), 417–439 (2022). <https://doi.org/10.2478/popets-2022-0021>
6. Baldimtsi, F., Chalkias, K.K., Madathil, V., Roy, A.: SoK: Privacy-preserving transactions in blockchains. *Cryptology ePrint Archive*, Paper 2024/1959 (2024), <https://eprint.iacr.org/2024/1959>
7. Ben Sasson, E., Chiesa, A., Garman, C., Green, M., Miers, I., Tromer, E., Virza, M.: Zerocash: Decentralized anonymous payments from Bitcoin. In: *2014 IEEE Symposium on Security and Privacy*. pp. 459–474 (2014). <https://doi.org/10.1109/SP.2014.36>
8. Bentov, I., Ji, Y., Zhang, F., Breidenbach, L., Daian, P., Juels, A.: Tesseract: Real-time cryptocurrency exchange using trusted hardware. In: *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*. p. 1521–1538. CCS ’19, Association for Computing Machinery, New York, NY, USA (2019). <https://doi.org/10.1145/3319535.3363221>, <https://doi.org/10.1145/3319535.3363221>
9. Bentov, I., Kumaresan, R.: How to use Bitcoin to design fair protocols. In: Garay, J.A., Gennaro, R. (eds.) *Advances in Cryptology – CRYPTO 2014*. pp. 421–439. Springer Berlin Heidelberg, Berlin, Heidelberg (2014)
10. Bowe, S., Chiesa, A., Green, M., Miers, I., Mishra, P., Wu, H.: Zexe: Enabling decentralized private computation. In: *2020 IEEE Symposium on Security and Privacy (SP)*. pp. 947–964 (2020). <https://doi.org/10.1109/SP40000.2020.00050>
11. Brickell, E., Li, J.: Enhanced Privacy ID: A direct anonymous attestation scheme with enhanced revocation capabilities. *IEEE Transactions on Dependable and Secure Computing* **9**(3), 345–360 (2012). <https://doi.org/10.1109/TDSC.2011.63>
12. Bünz, B., Agrawal, S., Zamani, M., Boneh, D.: Zether: Towards privacy in a smart contract world. In: Bonneau, J., Heninger, N. (eds.) *Financial Cryptography and Data Security*. pp. 423–443. Springer International Publishing, Cham (2020)
13. Castejon-Molina, D., Vasilopoulos, D., Moreno-Sanchez, P.: MixBuy: Contingent payment in the presence of coin mixers. *Proceedings on Privacy Enhancing Technologies* **2025**(1), 671–706 (2025). <https://doi.org/10.56553/popets-2025-0036>

14. Chen, G., Chen, S., Xiao, Y., Zhang, Y., Lin, Z., Lai, T.H.: SgxPectre: Stealing Intel secrets from SGX enclaves via speculative execution. *IEEE Security Privacy* **18**(3), 28–37 (2020). <https://doi.org/10.1109/MSEC.2019.2963021>
15. Cheng, R., Zhang, F., Kos, J., He, W., Hynes, N., Johnson, N., Juels, A., Miller, A., Song, D.: Ekiden: A platform for confidentiality-preserving, trustworthy, and performant smart contracts. In: 2019 IEEE European Symposium on Security and Privacy (EuroS&P). pp. 185–200. IEEE (2019)
16. Choudhuri, A.R., Green, M., Jain, A., Kaptchuk, G., Miers, I.: Fairness in an unfair world: Fair multiparty computation from public bulletin boards. In: Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security. p. 719–728. CCS '17, Association for Computing Machinery, New York, NY, USA (2017). <https://doi.org/10.1145/3133956.3134092>, <https://doi.org/10.1145/3133956.3134092>
17. Das, P., Ekey, L., Frassetto, T., Gens, D., Hostáková, K., Jauernig, P., Faust, S., Sadeghi, A.R.: FastKitten: Practical smart contracts on Bitcoin. In: 28th USENIX Security Symposium (USENIX Security 19). pp. 801–818. USENIX Association, Santa Clara, CA (Aug 2019), <https://www.usenix.org/conference/usenixsecurity19/presentation/das>
18. Dotan, M., Tochner, S., Zohar, A., Gilad, Y.: Twilight: A differentially private payment channel network. In: 31st USENIX Security Symposium (USENIX Security 22). pp. 555–570. USENIX Association, Boston, MA (Aug 2022), <https://www.usenix.org/conference/usenixsecurity22/presentation/dotan>
19. Dziembowski, S., Ekey, L., Faust, S.: FairSwap: How to fairly exchange digital goods. In: Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security. p. 967–984. CCS '18, Association for Computing Machinery, New York, NY, USA (2018). <https://doi.org/10.1145/3243734.3243857>, <https://doi.org/10.1145/3243734.3243857>
20. Erwig, A., Faust, S., Riahi, S., Stöcker, T.: CommiTEE : An efficient and secure commit-chain protocol using tees. In: 2023 IEEE 8th European Symposium on Security and Privacy (EuroS&P). pp. 429–448 (2023). <https://doi.org/10.1109/EuroSP57164.2023.00033>
21. Frassetto, T., Jauernig, P., Koisser, D., Kretzler, D., Schlosser, B., Faust, S., Sadeghi, A.: POSE: Practical off-chain smart contract execution. In: 30th Annual Network and Distributed System Security Symposium (NDSS). The Internet Society (2023). <https://doi.org/10.14722/ndss.2023.23118>
22. Hopwood, D.E., Bowe, S., Hornby, T., Wilcox, N.: Zcash protocol specification, version [NU5] (2024), <https://zips.z.cash/protocol/nu5.pdf>
23. Jean-Louis, N., Li, Y., Ji, Y., Malvai, H., Yurek, T., Bellemare, S., Miller, A.: SGXonerated: Finding (and partially fixing) privacy flaws in TEE-based smart contract platforms without breaking the TEE. *Proceedings on Privacy Enhancing Technologies (PoPETs)* **2024**(1), 617–634 (2024). <https://doi.org/10.56553/popets-2024-0035>, <https://petsymposium.org/popets/2024/popets-2024-0035.php>
24. Juels, A., Kosba, A., Shi, E.: The Ring of Gyges: Investigating the future of criminal smart contracts. In: Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security. p. 283–295. CCS '16, Association for Computing Machinery, New York, NY, USA (2016). <https://doi.org/10.1145/2976749.2978362>, <https://doi.org/10.1145/2976749.2978362>
25. Kaptchuk, G., Miers, I., Green, M.: Giving state to the stateless: Augmenting trustworthy computation with ledgers. In: Network and Distributed Systems Security (NDSS) Symposium. Internet Society (2019)

26. Kosba, A., Miller, A., Shi, E., Wen, Z., Papamanthou, C.: Hawk: The blockchain model of cryptography and privacy-preserving smart contracts. In: 2016 IEEE Symposium on Security and Privacy (SP). pp. 839–858 (2016). <https://doi.org/10.1109/SP.2016.55>
27. Kumaresan, R., Moran, T., Bentov, I.: How to use Bitcoin to play decentralized poker. In: Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security. p. 195–206. CCS '15, Association for Computing Machinery, New York, NY, USA (2015). <https://doi.org/10.1145/2810103.2813712>, <https://doi.org/10.1145/2810103.2813712>
28. Lee, J., Kim, S., Park, S., Moon, S.M.: RouTEE: Secure, scalable, and efficient off-chain payments using trusted execution environments. In: Proceedings of the Annual Computer Security Applications Conference (ACSAC) (2024), <https://www.acsac.org/2024/program/final/s161.html>
29. Lind, J., Eyal, I., Pietzuch, P., Sirer, E.G.: Teechan: Payment channels using trusted execution environments (2017), <https://arxiv.org/abs/1612.07766>
30. Lind, J., Naor, O., Eyal, I., Kelbert, F., Sirer, E.G., Pietzuch, P.: Teechain: a secure payment network with asynchronous blockchain access. In: Proceedings of the 27th ACM Symposium on Operating Systems Principles. p. 63–79. SOSP '19, Association for Computing Machinery, New York, NY, USA (2019). <https://doi.org/10.1145/3341301.3359627>, <https://doi.org/10.1145/3341301.3359627>
31. Möser, M., Böhme, R., Breuker, D.: Towards risk scoring of Bitcoin transactions. In: Financial Cryptography and Data Security. pp. 16–32. Springer (2014)
32. Möser, M., Narayanan, A.: Effective cryptocurrency regulation through blacklisting (2019), <https://maltemoeser.de/paper/blacklisting-regulation.pdf>, manuscript
33. Pass, R., Shi, E., Tramèr, F.: Formal abstractions for attested execution secure processors. In: Coron, J.S., Nielsen, J.B. (eds.) Advances in Cryptology – EUROCRYPT 2017. pp. 260–289. Springer International Publishing, Cham (2017)
34. Pfitzmann, A., Köhntopp, M.: Anonymity, Unobservability, and Pseudonymity — A Proposal for Terminology, pp. 1–9. Springer Berlin Heidelberg, Berlin, Heidelberg (2001). https://doi.org/10.1007/3-540-44702-4_1
35. PwC: PwC global crypto regulation report 2025 (2025), <https://legal.pwc.de/en/services/pwc-legals-eu-regulatory-compliance-operations/pwcs-global-crypto-regulation-report>
36. Secret Foundation: Secret Network graypaper: The foundation of a confidential decentralized future (2020), <https://scret.network/graypaper>, accessed: 4 November 2025
37. Thyagarajan, S.A., Malavolta, G., Moreno-Sanchez, P.: Universal atomic swaps: Secure exchange of coins across all blockchains. In: IEEE Symposium on Security and Privacy (SP). IEEE (2022)
38. Thyagarajan, S.A., Malavolta, G., Schmid, F., Schröder, D.: Verifiable timed linkable ring signatures for scalable payments for Monero. In: Atluri, V., Di Pietro, R., Jensen, C.D., Meng, W. (eds.) Computer Security – ESORICS 2022. pp. 467–486. Springer Nature Switzerland, Cham (2022)
39. Van Bulck, J., Minkin, M., Weisse, O., Genkin, D., Kasikci, B., Piessens, F., Silberstein, M., Wenisch, T.F., Yarom, Y., Strackx, R.: Foreshadow: extracting the keys to the intel SGX kingdom with transient out-of-order execution. In: Proceedings of the 27th USENIX Conference on Security Symposium. p. 991–1008. SEC'18, USENIX Association, USA (2018)

40. Van Schaik, S., Seto, A., Yurek, T., Batori, A., AlBassam, B., Genkin, D., Miller, A., Ronen, E., Yarom, Y., Garman, C.: SoK: SGX.Fail: How Stuff Gets eXposed. In: 2024 IEEE Symposium on Security and Privacy (SP). pp. 4143–4162 (2024). <https://doi.org/10.1109/SP54263.2024.00260>
41. Wilde, A., Gruel, T.N., Soriente, C., Karame, G.: The forking way: When TEEs meet consensus. In: Network and Distributed System Security (NDSS) Symposium. Internet Society, San Diego, CA, USA (February 2025). <https://doi.org/10.14722/ndss.2025.241934>
42. Wüst, K., Matetic, S., Schneider, M., Miers, I., Kostiaainen, K., Čapkun, S.: ZLiTE: Lightweight clients for shielded Zcash transactions using trusted execution. In: Goldberg, I., Moore, T. (eds.) Financial Cryptography and Data Security. pp. 179–198. Springer International Publishing, Cham (2019)